Linguaggi Per La Rete

Titotto Paolo
Appunti del Prof. Albarelli

Indice

1 ESPRESSIONI **3**

2 TIPI **4**

3 FUNZIONI E OGGETTI **5**
3.1 **PROTOTYPE** 7
3.2 **FUNZIONI** 9
3.3 **fFUNZIONI SECONDO ORDINE** 11
3.4 **UTILIZZO FUNZIONI COME METODI** 14

4 OPERATORI **14**

5 ARRAY **15**

6 ITERATORI **15**

7 CLASSI **20**
7.1 **FUNZIONI STATICHE** 22
7.2 **SUBCLASSING** 24
7.2.1 **NUMERI COMPLESSI - CAMPI PRIVATI** 26

8 WEB **26**
8.1 **DOM** 26
8.2 **EVENTI** 29
8.3 **SVG E CANVAS** 30

1 ESPRESSIONI

Linguaggio interpretato => no legato a linguaggio macchina

GESTIONE MEMORIA, VARIABILI, TIPI E ESPRESSIONI SONO INVARIATI

ESPRESSIONI-> stringhe di testo valutate. Inserire qualcosa che viene compilato in un valore->dipende dal linguaggio e viene immagazzinato in qualche modo.

```
<script>
    let a  = 1; //let manda in un dizionario
    let b = 2;
    a = b;
    a = 3;
    a = 1.2;
    a = 'test';
</script>
```

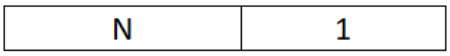


Figure 1: creazione di un dizionario

```
int main(){
    int a = 1;
    int b = 2;
    a = b;
    a = 3;
    a = 1.2;
    a = 'test';
}
```

ESPRESSIONE LETTERALE->producono un valore.

Linguaggi possono caricare valori dentro il programma. Linguaggi compilati producono blocchi mentre con quelli interpretati mantengo i tipi, vengono TRASFORMATI IN UNA COPPIA CHIAVE-VALORE IN BASE ALLA FORMA DEL LETTERALE, SONO INDIVISIBILI E VANNO NELLO HEAP. NON SAPPIAMO LA RAPPRESENTAZIONE. ALLOCATE NELLO GLOBAL SCOPE. Ho sempre il nome e il tipo della variabile.

Non ho il controllo a compilazione e il concetto di stack. Queste sono risorse dei linguaggi compilati

```
let a = 1;
let b = 10;
let x = b ?? a;
let y = 1;
y = a = 3, b = 7; //in questo caso y = 3 per questioni
//di priorit  e devo mettere le parentesi per fare y = 7
let c = a;
var d;
let t = -1;
if(t){
    c =  "pippo";
}
a = a + true;
d = 10;
console.log(typeof a);
```

2 TIPI

```
<script>
    let a; //istruzione
    a = 1;
    a = a + '+';
    let b = a > 5;
</script>
```

Si sono creati subito gli spazi per il dizionario **GLOBAL**.
Javascript moderno ha solo istruzioni con side effect.

'a'	N-> S	1-> '+'
'b'	B	'+' > 5

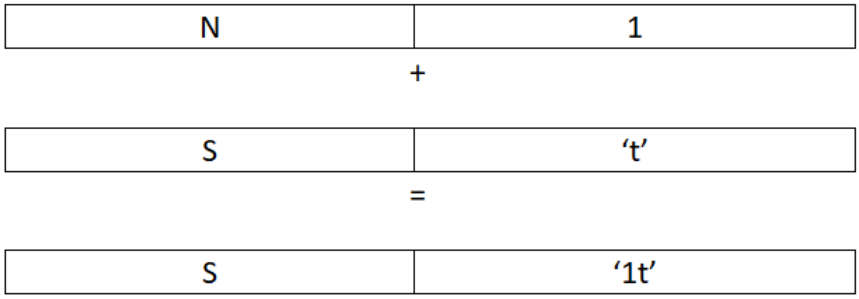
```
int main(){
    int a; //espando stack in base al tipo
    a = 1; //viene spostato lo StackPointer
    //quando mi muovo nell'esecuzione del programma
}
```

LET INSERISCE VARIABILI NEL DIZIONARIO CON NOME, TIPO E VALORE => U U = UNDEFINED CHE VIENE MESSO DI DEFAULT.
Possiamo dare meno parametri del necessario e gli altri vengono caricati di

'a'	U	U

default quando chiamo una funzione.

STRINGA COME TIPI NATIVI = IMMUTABILE.
OPERATORE '=' METODO -> sono polimorfi => fanno cose diverse in base ai tipi, fanno ciò che gli è stato scelto di fare, gli si può far fare tutto con quasi tutto.
Si vede: quindi ne crea la CONCATENZIONE



Se faccio la somma tra un numero e None otterrò sempre None.

Sovrascrivo sempre quando modifico il valore di una variabile.
Se dimentico LET in una funzione non inserisce variabili nel dizionario della funzione ma nel global e posso aver sovrascritto e posso usare valori errati. Non si inseriscono mai variabili nel global se non quelle di default.

USESTRIC E' DIRETTIVA-> comunica se sto facendo errori e se sto usando nuovo codice.
TIPI ATOMICI/NON COMPOSTI SONO IMMUTABILI. Sono:

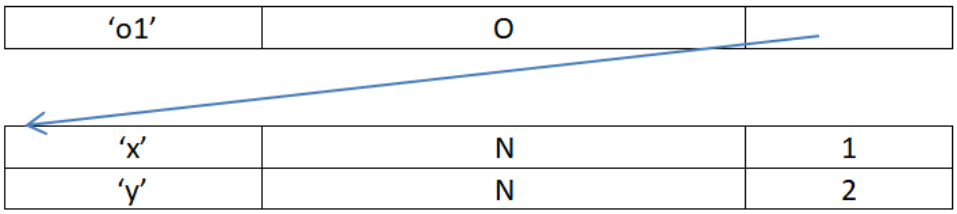
- UNDEFINED
- NUMBER
- STRING
- BOOLEAN
- BIG INT-> no floating point ma intero a precisione arbitraria let c = 1234m.

3 FUNZIONI E OGGETTI

OGGETTI = let o1 = {x:1, y:2} dove x è il nome del campo e 1 il valore. Quando creo un oggetto devo per forza usare i letterali.

Gli oggetti possono essere innestati => fondamentale l'uso delle {}.

Viene creato un nuovo dizionario con una reference al vecchio.



NON POSSO FAR PUNTARE AD ALTRO MA GLI ELEMENTO DELL'OGGETTO SONO MUTABILI. Tutto sta nello heap e non ordinato e neanche i dizionari sono ordinati perchè sono hashmap.

Esempio:

```
<html>
    <body>
        <script src = "test.js"></script>
    </body>
</html>
```

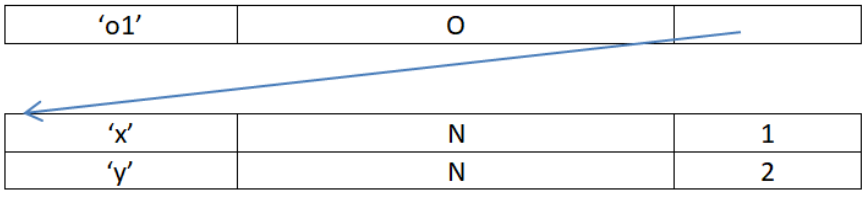
Importante la sequenza perchè script usati per codice pagina e creare effetti collaterali => rilasciare variabili dovute a una libreria.

Lo scope è un thread che corrisponde a una singola pagina, ma uno più script possono causare effetto collaterale che devo gestire.

JS NON HA UN ENTRY POINT.

Non esistono le classi come in Java => per avere funzioni statiche o COMODE LE METTO DENTRO UN OGGETTO PERCHE' ANCHE LE FUNZIONI SONO VARIABILI quindi sono nel global.
VAR va subito nel global.

let o1 = x:1, y:2, ha la stessa notazione del json. Nel global crea una variabile. Oggetto punta a un dizionari con il contenuto dell'oggetto => E' UNA REFERENCE DOVE LET O2 = O1 FA UNA COPIA.



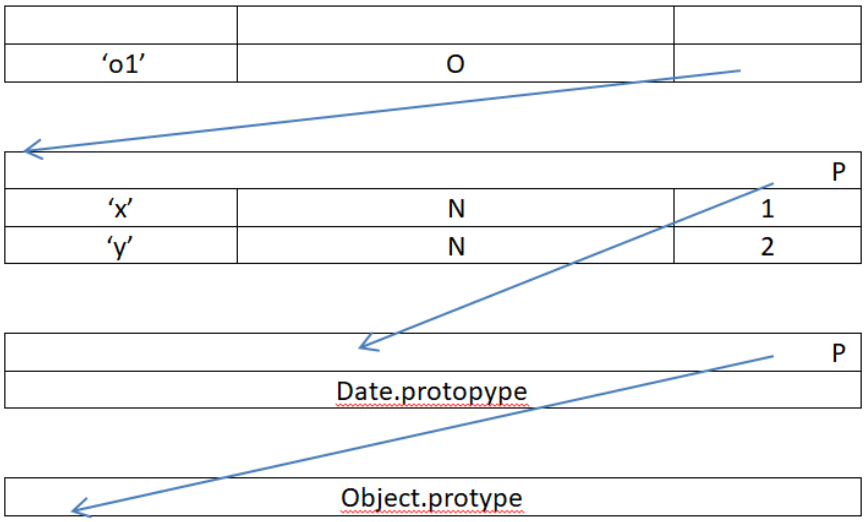
O1['X'] = O1.X-> identificativo. Nel primo caso uso la stringa dove calcolo il valore identificativo. Se faccio o1.z = 3 mi crea il campo z se non esiste.

- LET O1 = JSON.PARSE(S) dove json è un oggetto. let S = "a:1, b:2"
- JSON.STRINGFY(O3) l'output è rappresentato in stringa.

3.1 PROTOTYPE

d1 = new Date() ha un prototype. Date ha tipo f ed è una funzione che sta nel global.

OGGETTO HA ASSOCIATO UN ALTRO OGGETTO CHE E' IL SUO PROTOTYPE -> NON MODIFICABILE -> OGGETTO NEL GLOBAL-> TIPO.PROTOTYPE. Legato al costruttore.



OGNI OGGETTO CARATTERIZZATO DA UN PROTOTYPE

PROTOTYPE NO CAMPO DI UN OGGETTO ma assegnato alla creazione dell'oggetto. Se letterale è object.prototype altrimenti è il nome del costruttore CHIAMANDO FUNZIONE DETTA FACTORY -> COSTRUISCE QUALCOSA MA NO COSTRUTTORE=> statica che produce qualcosa. Al suo interno parametro per specificare il tipo del suo prototype.
OBJECT.PROTOTYPE E' OGGETTO => DIZIONARIO MA NEL GLOBAL SENZA PROTOTYPE. Tutti gli altri puntano a lui => simula EREDITARIETA' CLASSI.
I prototype servono per accesso ai campi degli oggetti.
Se un campo non viene trovato RIPERCORRE TUTTA LA CATENA DEI

PROTOTYPE, meccanismo call back per accesso ai campi per lettura.
Meccanismo scrittura avviene precocemente quindi se non trovo campo nel mio oggetto lo inserisco.
Il tutto simula override classi.
METODI FUNZIONI INTERNE AGLI OGGETTI E FUNZIONI SONO VALORI COME CAMPI, stesso funzionamento anche per il prototype. Non esiste il dispatch.

OGGETTO HA CAMPI PROPRI E NON (COLLEGATI AL PROTOTYPE) quindi per distinguerli uso funzioni:

- O5.HASOWNPROPERTY("X") se campo prorpio true altrimenti false.
- ACCESSO CONDIZIONATO LET O1 = O5?.X restituisce valore x se lo trova nella catena altrimenti lo definisce.
- OPERATORE SPREAD LET O5 = Z:1...O1 prende tutte le coppie o1 e le numera
- VALUEOF restituisce valore primitivo che rappresenta oggetto

```
let o1 = {x : 3, y : -2, tst : true};
let toStringFunction = o1.toString;
let stringValue = o1.toString();
let k = toStringFunction();
let o2 = {a : "test a", b : "test b"};
o2 = o1;
let o3 = Object.create(o1);
o3.y = "My y"
let v1 = o3.y;
let v2 = o3.x;
let v3 = o3.z;

let f1 = function(a, b){
    let max = a;
    if(b > max)
        max = b;
    return max;
};
```

3.2 FUNZIONI

```
let StringValue = o1.toString;
//accede al campo toString che e' nel global
//StringValue ha tipo funzione perche'
//assegno toString che    una funzione

let c = o1.toString(); //invocazione metodo
```

Sono entrambi accessi a campi-> va nel prototype perchè o1 non ha il campo toString.

FUNZIONE E' UN OGGETTO-> posso accedere ai campi e sono un tipo nativo.

Ci sono SCOPE nelle funzioni oltre al prototype come gli oggetti. Formano una catena che risalgo per accedere ai campi.

Posso creare funzioni senza () assegnandola a una funzione/campo di un oggetto senza mettere le (). La posso invocare usando le () perchè il nome che c'è nell'hashmap comunica con la funzione che ho nello heap.

LENGTH-> numero parametri espliciti che si aspetta esempio anzichè fare sun(a,b) si può fare sum(1,2) che da il nome ai parametri.

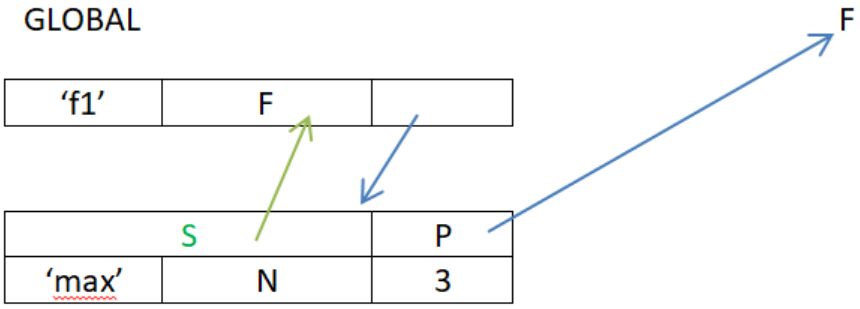
PARAMETRI NON FANNO PARTE DELLA FIRMA DELLA FUNZIONE no perchè funzione è un parametro => no override.
Posso concatenare funzioni.

Definibili in 3 modi:

- DEFINIBILE CON LETTERALE DI FUNZIONE

  ```
  //letterali che producono cose
  let a = 3;
  let o = {x:2, y:2};
  let f1 = function(a, b){ //tipo reference
              let max = a;
              if(b > max)
                  max = b;
              return max;
          };
  ```

  Sta nello heap. Hanno prototype ed E' UN OGGETTO f1.c = 3 INSERISCE NUOVO CAMPO perchè ho numero infinito di parametri che posso esplicitare.



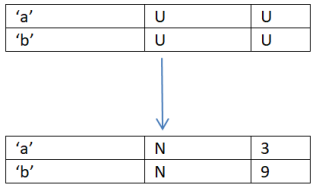
FUNZIONE CONTIENE POTENZIALMENTE CAMPI

ASSEGNAMENTO CAMPO A FUNZIONE COME COSTRUTTORE HA EFFETTO IMPORTANTE E POSSO CAMBIARE PARAMETRI PER CAMBIARE COMPORTAMENTO

FUNZIONE HA CODICE

SCOPE-> rende disponibili variabili accesso diretto ed è un oggetto implicito.
SUPERSCOPE-> puntatore allo scope attivo (uno solo per ogni momento) quando è stata creata la funzione.
PROTOTYPE-> usato per settare automaticamente i prototype degli oggetti creati con quella funzione come costruttore.

f1(3, 9) invocazione => crea nuovo scope locale e assegna valori. f1 operatore accesso variabili e () operatore eseguito sulle variabili.



+ oggetto che contiene un array con i parametri, creato in automatico.

Viene assegnato come SUPERSCOPE QUELLO DOVE E' STATA DEFINITA e gli sono accessibili anche i campi.

- ```
  function min(c, d){
      if(c > d)
          return d;
      else
          return c;
  }
  ```

  Differenze:

  1. Abilitato HOISTING-> funzione non disponibile finchè non è stata creata.
  2. Ha un nome ma soprattitto è AUTOMATICAMENTE CONST QUELLA CREATA CON IL LETTERALE

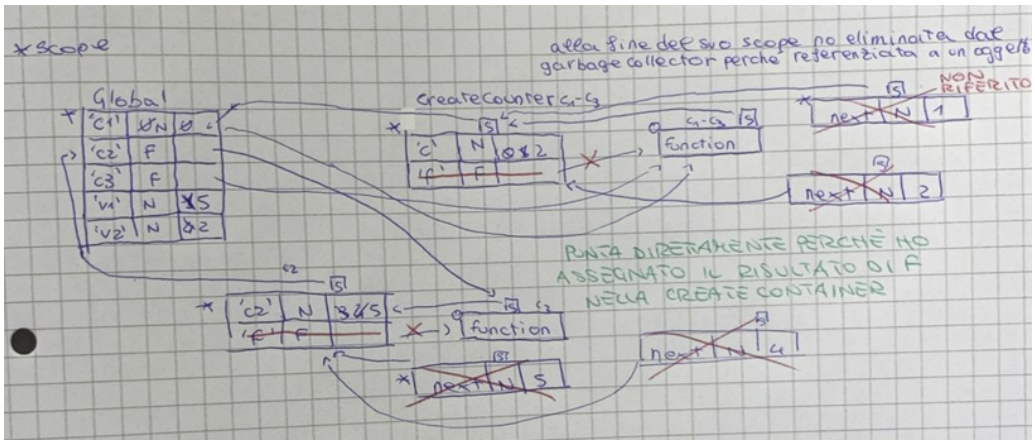
3.3 fFUNZIONI SECONDO ORDINE

Funzione che prende scope di una funzione esistente.

```
function createCounter(start){
    let c = start ?? 0;
    //cattura contenuto in cui e' stata creata => c
    //Non hanno vero nome, varia dalla variabile
    //perche' non sono indirizzi di memoria
    let f = function(){
        let next = c + 1;
        c = next;
        return next;
    };
    return f; //ritorna il metodo
}
let c1 = createCounter(0); //se non metto nulla, valore
//default o undefined
let c2 = createCounter(3);
let c3 = c1;
let v1 = c1();
let v2 = c2();
v1 = c2();
v2 = c3();
```

let c1 = createCounter(0); e let c2 = createCounter(3); sono FUNZIONI.

let v2 = c2(); e v1 = c2(); e v2 = c3(); sono VALORI DETERMINATI DALLA FUNZIONE. () OPERATORE DI FUNZIONE.
```



CHIUSURA -> mantiene solo variabili e funzioni che vengono utilizzate e le altre le toglie. CREA AMBIENTE NON GLOBALE NE' LOCALE MA CONDIVISO.
OGNI INVOCAZIONE A FUNZIONE CREA UN NUOVO SCOPE.

c1 e c2 creano scope temporanei ma uguali.
Se metto c2 = 7 modifico il global e si toglie tutto c2 come scope e oggetto.
Ottengo un metodo con campi associati.
CHIUSURE CONDIVISE TRA PIU' FUNZIONI

PROXY -> accedere contenuto pagina e oggetti.

Le linee prima dello script in html vengono converite in oggetti javascript mentre quelle dopo non è detto.

```
function button(text, func){ // funzione d'appoggio per
//creare i bottoni
    let button = document.createElement("input");
    button.setAttribute("type","button");
    button.setAttribute("value", text);
    button.onclick = func; // associo la funzione voluta a
    //onclick
    return button;
}


function createGallery(g){ // g corrisponde al div
//dell'html, oggetto
    let images = g.getElementsByTagName("img"); // estrae
```

```
    //le immagini dall'html come fosse una
    //collezione di immagini
    //prototype htmlimageElement.prottype
    let s = []; // salvo i sorgenti delle immagini array

    while(images.length > 0){
        s.push(images[0].getAttribute("src"));
        g.removeChild(images[0]); // rimuove l'immagine
        //da images e lo ritorna
    }

    let img = document.createElement("img"); // crea
    //l'elemento img nel proxy
    let i = 0;
    img.setAttribute("src",s[i]); // imposto l'src al
    //primo elemento delle
    //immgini precedentemente tolte
    //in chiusura tengo i che e' indice imm e s che le
    //contiene a anche img per gli eventi = gestore

    g.appendChild(button("<-",function(){ // aggiunge
    //all'html l'immagine con
    //le freccia indietro
        img.setAttribute("src",
        s[i = i - 1 >= 0 ? i-1 : s.length-1]);
    }));

    g.appendChild(img);

    g.appendChild(button("->",function(){
    // aggiunge all'html l'immagine con le freccia avanti
        img.setAttribute("src",s[i = (i + 1) % s.length]);
    }));
}

let galleries = document.getElementsByClassName("gallery");
// gallery
//   il nome dato alla classe nell'html
for(let gallery = 0; gallery < galleries.length; gallery++)
    createGallery(galleries[gallery]);
// a questo punto tutte le immagini dei div sono stati
//rimossi
```

let images = g.getElementsByTagName("img") immagini vengono IMMAGAZZINATE IN OGGETTO VIVO => più specifico di un array che se elimino un elementoda albero che si crea si toglie anche dall'array.

Metodi per rimuovere = 3 output:

- ELIMINA
- TRUE/FALSE
- RESTITUIRE OGGETTO RIMOSSO

3.4 UTILIZZO FUNZIONI COME METODI

```
let square = {  //OGGETTO CON 2 PROPERTY
    side: 10,
    area: function(sideValue){retuturn sideValue*sideValue;}
    //FUNCTION
}
let areaEvaluationFunction = square.area; //ASSEGNA FUNZIONE
//A LETTERALE CHE DIVENTA TIPO FUNZIONE
Llet myArea = areaEvaluationFunction(square.side);
//INVOCA FUNZIONE E RESTITUISCE
myArea = sqaure.area(square.side);
```

Le due invocazioni sono diverse perchè quando chiamo funzione in modo normale e non come metodo compare THIS = WINDOW mentre se faccio direttamente con invocazione THIS = OBJECT => OGGETTO DA CUI E' APPENA STATA ESTRATTA LA FUNZIONE-> posso usare direttamente this.side => il valore che mi è stato passato, se this non metto nulla nell'invocazione NON POSSO METTERE THIS perchè variabile locale.

4 OPERATORI

- TYPEOF: console.log(typeof a) dice il tipo della variabile e sono tutti considerati stringhe.
- ??: let x = b ?? a indica che se b ha valore decente assegna b altrimenti a.
- LET Y = 3, X = 7: tratta come unica espressione e mantiene ultima fatta (x = 7)
- , OPERATORE CONTINUATION: fa gli assegnamenti con il suo valore, fa tutto e vale ultima cosa fatta.

5 ARRAY

Non esistono, no continui in memoria ma sono OGGETTI CON INDICI CHE SONO NUMERI MA TRASFORMATI IN STRINGA.

```
let x1 = {};
x1[0] = 7;
x1[1] = 3;
let v = x1["0"] = 7; //perch  trasforma in stringa
```

'1'	N	3
'2'	N	7

Tempo di accesso virtalmente O(1) perchè tabella hash.
Sconosciuto a livello nativo ma supporto dato dal prototyping con alcuni metodi, bisogna chiamarlo let x1 = new Array(). x̀length() la trova nel prototype array.prototype.

FUNZIONE ESPRIMIBILI CON LAMBDA. Posso dare valore o causare un effetto collaterale non ritornando.

...-> espande qualsiasi tipo iterabile trasformandolo in un array. Anche le stringhe sono iterabili.
let a = [..."Hello World"] crea array 11 posizioni.
ARGUMENT -> array che contiene parametri formali funzione.

... in una funzione-> mette variabili in un array con il nome messo di fianco.

6 ITERATORI

2 TIPI ITERATORI:

- FOR OF -> for(letter of letters) carica elementi dell'iteratore
- FOR IN -> oiù a basso livello e lavora guardando oggetto e i suoi campi. ITERA NELLE CHIAVI => prende tutti i campi. PASSA ANCHE LE FUNZIONI DEI MIEI OGGETTI E NO SOLO CAMPI.

  [x, y] = [y, x] swap variabili array => può essere left value.
  ENTRIES -> metodo array, nuovo proxy con coppie chiave-valore -> mostra iteratore.
  for(let [index, letter] of letters.entries)) mi da indice e valore.
  Posticipare concatenazione elementi provavilmente accederò in maniera

sequenziale.

- FOREACH -> prende funzione e la esegue su tutti gli elementi della lista su cui lo eseguo, viene passato il parametro su cui devo eseguire, posso passargli anche una funzione => gli passo cosa deve fare e può avere più parametri.

  FUNZIONE ANONIMA -> forEach(value)=>sum += value;.
  Può rocevere indici e array e valori data.forEach(v. i, a) =>a[i] = v + 1;.
  REDUCE()-> passa anche processing parziale che gestisce lui.
  MAP() -> prendo funzione con valore e restituisce qualcosa e la scrive nell'elemento dell'array in quella posizione ma creando un nuovo array.

Parametri array -> elemento corrente, indice e array stesso.
ForEach si basa su EFFETTO COLLATERALE mentre map si basa su VALORE DI RITORNO delle funzioni.

FUNZIONE DEFINITA COME UN METODO LA VARIABILE SU CUI VIENE INVOCATA VIENE VALORIZZATA A THIS.

ARRAY.PROTOTYPE FA OVERRIDE DI OBJECT.PROTOTYPE-> unica cosa che si può fare tramite meccanismo del prototype. No overloading perchè parametri non fanno parte della firma delle fuzioni perchè sono CONTENUTO DELL'OPERTORE DI INVOCAZIONE.
Classi -> veicoli funzioni => oggetti uniformi alla loro struttura.
Queste funzioni non modificano array ma ne fanno una copia.

- FILTER-> prende funzione che ritorna booleano che decide se va inserita o meno nell'array.
- FINDINDEX-> trova primo INDICE array che rende vera l'espressione.
- EVERY-> restituisce un booleano e al primo false diventa false. let c = data.every(c=>c>0).
- REDUCE-> operazione aggregazione su array. passo parametri attuali valore aggregazione, valore attuale dell'array V, indice I e array A. data.reduce((current, v, i, a)=> current + v, 0) quindi anche punto partenza.

```
/*[x,y] = [y,x]; // scambia le variabili dell'array.
let letters = [..."Hello World"]; // array che contiene
11 elementi uno per ciascuna lettera
let reverse = "";
// for classico con gli indici
```

```
for(let letter = 0; letter<letters.length; letter++)
    reverse = letters[letter] + reverse;

// for..of Lavora sui valori degli iterable
for(let letter of letters){
    reverse = letter + reverse;
}

// for..in lavora sulle chiavi dell'oggetto. In questo
//caso ritorna gli indici di ogni elemento
for(let letter in letters){
    reverse = letter + reverse;
}

let everyother = "";
// entries() produce un iterable di coppie chiave-valore
//e mostra solo l'iteratore
for(let [index,letter] of letters.entries())
    if(index % 2 == 0) everyother += letter;

let o = {x:1, y:2};
for(let field of Object.keys(o)) // keys() metodo statico
//che torna un array contenti le chiavi di un oggetto
    field = field + 1;

*/

// PROGRAMMAZIONE CON METODI FUNZIONALI
let data = [1,2,3,4,5];
let sum = 0;
data.forEach(function(value){
    sum += value;
}); // prende una funzione e la esegue su tutti gli
//elementi dell'array(in generale dell'iterable)

// pu  essere convertita in:
data.forEach((value) => {sum += value;});
// utilizzo della lambda

let workingFunction = (v,i,a) =>
{console.log("ciao " + i + " " + v + " " + a);
return a[i] = v*v};
data.forEach(workingFunction);

let datasquared = data.map(x => x*x);
// crea un nuovo array con i risultati della funzione passata
```

```
//come parametro

// possibile implementazione di map
function mapArray(array, worker){
    let result = [];
    for(let i = 0; i < array.length; i++)
        result.push(worker(array[i],i,array));
    return result;
}

let b = data.filter(x => x < 3);
// se true inserisce l'elemento in b altrimenti va al
//successivo
let i = data.findIndex(x => x < 3);
// trova l'indice del primo elemento che rispetta la
//condizione
let e = data.find(x => x > 3);
// trova il primo elemento che rispetta la condizione
let allpositive = data.every(x => x > 0);
// restituisce true se tutti gli elementi rispettano la
//condizione
let somma = data.reduce((current,v,i,a) => current+v, 0);
// usata per operazioni di aggregazione
```

... MATERIALIZZA CONTENUTI ARRAY, esplode array.
Supporto nel linguaggio -> SPECIFICHE FUNZIONI CHE DANNO RISULTATI PARZIALI-> restituisce valori uno alla volta, si chiama YIELD che restituisce valore in alternativa al return, supportato dall'interprete.
Convezione per metodi speciali ITERATOR(). No nome specifico perchè no in classi ma devo metterlo all'interno di un oggetto.
Si è creato un ulteriore tipo SYMBOL che genera simboli anonimi freschi, simbolo che non esisteva. Oggetto e tipo.
Nell'iteratore esiste specifico simbolo che dice che è un iteratore.
SYMBOL.ITERATOR(). () serve per dire che è il nome di un metodo e no espressione. Iterator costante. Nome che passa per una costante anzichè nome.
FUNZIONE RESTITUISCE VALORI MULTIPLI => NO CHIUSA => SYMBOL.ITERATOR. Posso metterlo anche dopo il nome function*.

```
*[Symbol.iterator](){
// il suo scope rimane sempre attivo
//e la funzione non si chiude mai
    for(let x = Math.ceil(this.from); x < this.to; x++)
            yield x;
            // corrisponde ad un next() in Java.
            //Rimane in memoria solo x.
}
```

Ogni volta restituisce valore e la funzione non si chiude => non termina chiudendo lo scope, dovuto allo * e yield -> next in Java ma non esiste hasnext perchè termina fine funzione.
Scope termina quando finisce la funzione.
Crea uno scope ma non lo libera. Function call con scope esistente, possono essre richiamate usando scope vecchio. Anche program counter resta invariato.

```
class Range{
    constructor(from = 0,to = 0){
        if(typeof from == "number" && typeof to == "number"){
            this.from = from;
            this.to = to;
        } else if(typeof from == "string"){
            let parsedRange = Range.parse(from);
            this.from = parsedRange.from;
            this.to = parsedRange.to;
        } else {}
            //throw new TypeError("no good parameters");
    }
    *[Symbol.iterator](){
    // il suo scope rimane sempre attivo
    //e la funzione non si chiude mai
        for(let x = Math.ceil(this.from); x < this.to; x++)
                yield x;
                // corrisponde ad un next() in Java.
                //Rimane in memoria solo x.
    }

    includes(x){return this.from <= x && x < this.to;}
    toString() {return "(" +  this.from + "..."
    + this.to + ")";}
    static parse(s){
        let matches = s.match(/^\((\d+)\.\.\.(\d+)\)$/);
        if(!matches)
            throw new TypeError("Cannot parse " + s);
        return new Range(parseInt(matches[1]),
        parseInt(matches[2]));
        // prende i gruppi e le inserisce nel nuovo oggetto
    }
}

class Span extends Range{
    constructor(start, length){
        if(length < 0)
            throw new Error("negative length");
        super(start,start+length);
```

```
    }
}

let r1 = new Range(1,10);
let r2 = new Range(100,120);
let t1 = r1.includes(5);
let t2 = r2.includes(5);
console.log("" + r1);
console.log("" + r2);

let test = r1 instanceof Range;
let r3 = Range.parse("(1...10)");
let r4 = new Range("(20...34)");

// iteratori
let concreteRange = [...r4];

for(let v of r4)
// chiama ogni volta l'iteratore e ne restituisce l'elemento
//->FUNCTION CALL CON SCOPE GIA' ESISTENTE
    console.log("element is: "+v);
```

7 CLASSI

CLASSI-> struttura invocata come template e permette di non replicare codice.
MECCANISMO PROTOTYPING

DEFINIRE METODI COME GETTER E SETTER=> utilizzare metodi come quello LENGTH di array oppure mettere metodo read.only.

```
/*let square = {
    side : 10,
    area : function(sidevalue){
        let result = sidevalue;
        result = result * result;
        return result;
    }
   area(){return this.side * this.side;}
}

// TODO: scrivere come cambia this nei due casi
let areaEvaluationFunction = square.area;
let myArea = areaEvaluationFunction(square.side);
```

```
myArea = square.area(square.side);*/


let p = {
    x : 1.0,
    y : 1.0,

    get r(){return Math.hypot(this.x,this.y);},
    set r(v){
        let ratio = v / Math.hypot(this.x,this.y);
        // oppure let ratio = v / this.r; // invoca nel secondo caso il getter
        this.x *= ratio;
        this.y *= ratio;
    },

    get theta(){return Math.atan2(this.y,this.x);}
};

p.x = 4;
p.y = 3;
p.r = 10; // capisce che questo    un setter
let pr = p.r; // capisce che questo    un getter
let angle = p.theta;
//CREO NUOVO OGGETTO
let p1 = Object.create(p); // utlizza il meccanismo di prototype
let p2 = Object.create(p); // utilizza il meccanismo di prototype

let r1 = p1.r;
p2.r = 20;
let r2 = p2.r;
```

get r()return Math.hypot(this.x,this.y);, FUNZIONA SOLO IN LETTURA NO LVALUE, SE METTO GET POSSO INVOCARLA COME ESPRESSIONE RVALUE SENZA COMANDO INVOCAZIONE lui crea comunque normalmente ma dopo capisce che è un getter e esecuzione non cambia. POSSO INVOCARLA SENZA ()
Vantaggio livello refactor, ASTRAZIONE VALORI E CAMPI RISPETTO REALE IMPLEMENTAZIONE
con set devo ricevere input da esterno. Unico caso stessa firma e parametri diversi.
POSSO CREARE FUNZIONE TIPO FACTORY => STATICA PRODUCE OGGETO DEL TIPO CHE CI INTERESSA. Oggetto nuovo.

Metto tutti i metodi in un prototype campo della mia classe.
POSSO USARE NOME DELLA MIA CLASSE COME COSTRUTTORE CON KEYWORD NEW che automatizza creazione oggettoe standardizzare definizione

e proprietà.

Se dimenti new risolvo facendo PROPERTY NEW.TARGET CHE VALE FUNZIONE NEL CASO INVOCATA COME COSTRUTTORE E UNDEFINED ALTRIMENTI.
La metto in un controllo in modo da creare gli oggetti in modo corretto.

INSTANCEOF -> 2 operandi, classe controllare istanza e funzione usarla come costruttore, RITORNA BOOLEANO.
Controlla se Range costruttore r1 salvando costruttore oggetto e CONTROLLA SE PROTOTYPE OGGETTO UGUALE AL CAMPO PROTOTYPE DELLA FUNZIONE PASSATA COME PARAMETRO.
Risale catena prototype.
Controllo sanità del programma.

Altri modi per creare classe:
uso identificatico standard CLASS e dato nome crea lui il costruttore con lo stess nome e dentro lo stesso blocco, analoga anche con le funzioni. Costruttore uso nome CONSTRUCTOR, non devo mettere "," tra le funzioni perchè non sono campi. Esiste campo prototype ma è protetto da scrittura ma se provo a riscriverlo fallisce senza dire niente.
Rende codice STRICT, evita cose che non vanno bene.
NO RETURN NEI COSTRUTTORI

7.1 FUNZIONI STATICHE

Dichiaro come NUOVI CAMPI, let a = function().
SONO FACTORY. USO KEYWORD STATIC ma non metto Range.nome. Ritorna un oggetto.

REGEXPR EMBEDDED NEL LINGUAGGIO-> hanno un loro letterale let m = s.MATCH(/$)

```
/*function Range(from,to){
    let r = Object.create(Range.methods);
    r.from = from;
    r.to = to;

    r.includes = function(x){
        return this.from <= x && x < this.to;
    };
    return r;
}

// per metterlo in un protoype
Range.methods = {
```

```
    includes(x){return this.from <= x && x < this.to;},
    toString() {return "(" +
    this.from + "..." + this.to + ")";}
}



// Maniera standard con l'uso della keyword new
function Range(from,to){
    // let r = Object.create(Range.prototype);
    if(! new.target) // property per indicare se la funzione e'
    stata invocata come costruttore
        return new Range(from,to);

    this.from = from;
    this.to = to;

    // return r;
}

Range.prototype = {
    includes(x){return this.from <= x && x < this.to;},
    toString() {return "(" +  this.from + "..."
    + this.to + ")";}
}


function Other(){}
Other.prototype = Range.prototype;*/

class Range{
    constructor(from,to){
        if(typeof from == "number" && typeof to == "number"){
            this.from = from;
            this.to = to;
        } else if(typeof from == "string"){
            let parsedRange = Range.parse(from);
            this.from = parsedRange.from;
            this.to = parsedRange.to;
        } else
            throw new TypeError("no good parameters");
    }
    includes(x){return this.from <= x && x < this.to;}
    toString() {return "(" +  this.from + "..." + this.to
    + ")";}
    static parse(s){
```

```
        let matches = s.match(/^\((\d+)\.\.\.(\d+)\)$/);
        if(!matches)
            throw new TypeError("Cannot parse " + s);
        return new Range(parseInt(matches[1],matches[2]));
        // prende i gruppi e le inserisce nel nuovo oggetto
    }
}

let r1 = new Range(1,10);
let r2 = new Range(100,120);
let t1 = r1.includes(5);
let t2 = r2.includes(5);
console.log("" + r1);
console.log("" + r2);

let test = r1 instanceof Range;
let r3 = Range.parse("(1...10"));
let r4 = new Range("(1...10)");
```

7.2 SUBCLASSING

Si usa EXTENDS E CREO FUNZIONE CON NOME DELLA CLASSE E PROTOTYPE OGGETTO CON METODI DELLA CLASSE CHE HANNO COME PROTOTYPE QUELLO DELLA CLASSE CHE ESTENDONO. Analogo a overloading.
Posso chiamare SUPER-> chiamno costruttore e metodi della superclasse.
Esempio di sottoclassi:

```
class EZArray extends Array{
        get first(){
                return this[0];
        }
        get last(){
                return this[this.length-1];
        }
}

let a1 = new EZArray();
let a2 = new EZArray();
a1.push(2);
a1.push(4);
a1.push("pippo");
a2.push(true);
/*
console.log(a1);
console.log(a1.first);
```

```
console.log(a2.last);
console.log();

console.log(a1 instanceof EZArray);
console.log(a1 instanceof Array);
*/
class TypedMap extends Map{

        keyType = 'string';
        valueType = 'number';

        constructor(keyType,valueType,entries){

                super(entries);

                if(keyType)
                        this.keyType = keyType;
                if(valueType)
                        this.valueType = valueType;

                if(entries)
                        for(let [k,v] of entries)
                                this.checkTypes(k,v);
        }

        set(k,v){
                this.checkTypes(k,v);
                return super.set(k,v);
        }

        checkTypes(k,v){
        console.log(this instanceof TypedMap);
        if((this.keyType && typeof k !== this.keyType )||
           (this.valueType && typeof v !== this.valueType))
                throw "keyType must be "+this.keyType+
                " and valueType must be "+this.valueType;
        }
}

let tm = new TypedMap('string','boolean',[['a',true],['b',false]]);
console.log(tm.valueType);
//let defaultMap = new TypedMap('string','number');
//defaultMap.set('test',1);
```

7.2.1 NUMERI COMPLESSI - CAMPI PRIVATI

CLASS COMPLEX -> quando creo campi con il costruttore non sono privati e se faccio operazioni con Number ne crea uno nuovo => immutabili.
#r = 4 CAMPO PRIVATO, per modificarli devo sempre mettere . Sono inaccessibili perchè hanno aggiunto al campo il NUMER(#). Nell'identificatore è concesso il loro accesso sono nella classe e non all'esterno.

```
class Complex{
    static ZERO = new Complex(0,0);

    #r = 0; // campo privato
    #i = 0; // campo privato
    constructor(real = 0, imaginary = 0){
        this.#r = real;
        this.#i = imaginary;
    }

    plus(other){
        return new Complex(this.#r + other.#r,
        this.#i,other.#i);
    }

    static sum(a,b){return a.plus(b);}

    get i(){return this.#i;}
    get r(){return this.#r;}

    toString(){return this.r + "␣i"+ this.i;}
}


let c1 = new Complex(1,1);
let c2 = new Complex(2,3);
let c3 = c1.plus(c2);
let i_part = c3.i; // ottengo la parte immaginaria di c3
let c5 = Complex.sum(c1,c2);
console.log(c3);
```

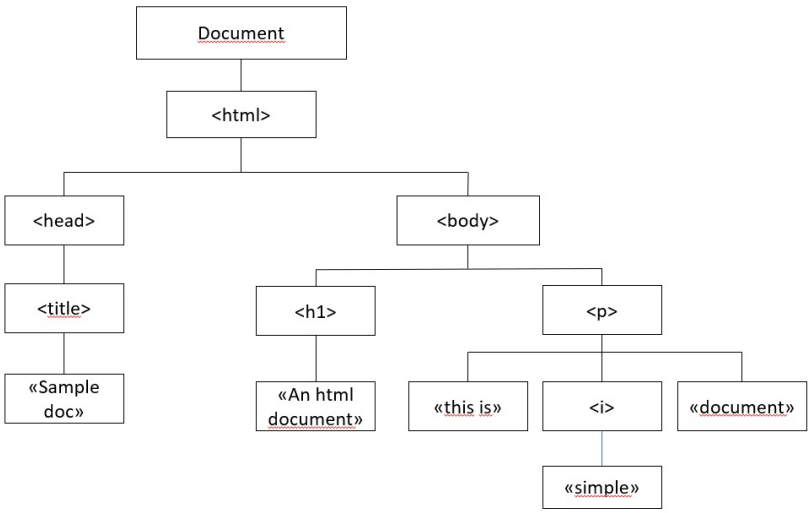
8 WEB

8.1 DOM

HTML tag innestati
Converito subito in pagina visuale

SEPRAZIONE OGGETTO REALE CHE BROWSER CREA. File testo -> deserializzato (trasformato struttura dati albero) dopo lettura-> mantenuto all'interno del motore nativo del browser
Javascript-> serie di classi DOCUMENT (DOM = DOCUMENT OBJECT MODEL) per interagire con elementi tramite PROXY
DOCUMENT-> radice principale non necessariamente HTML. All'interno ho vari nodi, tag stringhe e nodi => gerarchia. Include anche spazi, invii e tab. Nodi testo sono a blocchi e sono foglia
OGNI NODO HTML CORRISPONDE A UN OGGETTO PROXY JAVASCRIPT, oggetto prodotto con costruttore (definire prototyping) => elemento gerarchico di classi dei tipi proxy.
Operazioni che faremo per alterare documento:

- MODIFICARE CONTENUTO NODO
- CREARE NUOVO NODO
- MODIFICARE ATTRIBUTI
- NAVIGARE



Specializzazioni-> nodi con specifici comportamenti con metodi e attributi. ELEMENT = tag (headElement, bodyElement, ...), DOCUMENT = documento solitamente HTMLDOCUMENT e URL. CHARACTER DATA = COMMENT E TEXT, ATTRIBUTE = ad esempio il colore delle componenti come body
NODE LIST NON VIVA ritorna nodi no in una collection
GETCHILDREN danno info sui figli, da elemeti (figli tipo elements). CHILDNODES tutti fili tipo node.
MODIFICARE NODI -> ELEMENTI = setattribute e getattribute oppure riassegno CAMBIO STRUTTURA

Modifiche diposnibili al prossimo ciclo rendering => tipo gestore eventi.
Posso modificare testo TEXTCONTENT
INNERHTML permette di inserire codice HTML.

```
function button(text, func){
// funzione d'appoggio per creare i bottoni
    let button = document.createElement("input");
    button.setAttribute("type","button");
    button.setAttribute("value", text);
    button.addEventListener("click",func);
    // associo la funzione voluta all'evento di click
    return button;
}


function createGallery(g){
// g corrisponde al div dell'html
    let images = g.getElementsByTagName("img");
    // estrae le immagini dall'html come fosse
    //una collezione di immagini
    let s = []; // salvo i sorgenti delle immagini

    while(images.length > 0){
        s.push(images[0].getAttribute("src"));
        g.removeChild(images[0]);
        // rimuove l'immagine da images e lo ritorna
    }

    // a questo punto tutte le immagini dei div sono stati
    //rimossi

    let img = document.createElement("img");
    // crea l'elemento img nel proxy
    let i = 0;
    img.setAttribute("src",s[i]);
    // imposto l'src al primo elemento
    //delle immgini precedentemente tolte

    g.appendChild(button("<-",function(){
    // aggiunge all'html l'immagine con le freccia indietro
        img.setAttribute("src",s[i = i - 1 >= 0 ? i-1 : s.length-1]);
    }));

    g.appendChild(img); // aggiunge l'immagine all'html

    g.appendChild(button("->",function(){
```

```
    // aggiunge all'html l'immagine con le freccia avanti
     img.setAttribute("src",s[i = (i + 1) % s.length]);
    }));
}

let leftalignedDivs = document.querySelectorAll('div>img');
// ritorna tutti i div con quel attributo presenti nel documento
let scoreElement = document.getElementById("score");
scoreElement.textContent = "test";
var body = document.getElementsByTagName("Body")[0];
// ritorna un HTMLCollection con tutti
//gli elementi presenti nel body
let galleries = body.getElementsByClassName("gallery");
// gallery    il nome dato alla classe nell'html
//ricavabile anche dal body
for(let gallery = 0; gallery < galleries.length; gallery++)
    createGallery(galleries[gallery]);
```

8.2 EVENTI

LIFE CYCLE = serie step e ciclo gestione alla fine, canvas e VM.
Carica sorgente, primo step asincrono
Esecuzione script al caricamento pagina , script creano EFFETTI COLLATERALI (AGGANCIARE GESTIONE EVENTI) creano variabili nel global.
Richiamato dal thread della pagina
Rendering pagina
Gestione eventi, iterazione SO e device quindi nascono da SO. Codice c++ browser si connette a gestore eventi dispositivi. Li spacchetta e li trasforma in OGGETTO EVENTO, coda unica per la pagina. Ordinamento eventi e timestamp
Thread vede oggetto su cui stato eseguito e se ha GESTORE PER EVENTO funzione unico parametro evento. Se presente viene eseguita e aggiugne evento alla coda. Modifica pagina unico thread.
OGGETTO EVENTO

- CAMPO TYPE = distingue evento, string
- CAMPO TARGET = oggetto che ha ricevuto evento, this, DOM. No elemento a cui associata funzione perchè eventi propagati a elementi che lo contengono.
- CURRENT TARGET = elemento dell'evento attuale

Se dal DOM vedo elementi con evento CAPTURING vengono gestiti prima se catturati. Propagazione dall'alto al basso.
Posso bloccare con metodo STOP PROPAGATION ferma propagazione. PREVENT DEFAULT gestore browser, no blocco propagazione ma browser non

carica altra pagina. REMOVEEVENTLISTENER ha bisogno di conoscere la funzione.

```
let innerElement = document.getElementById("inner");
let outerElement = document.getElementById("outer");
let goElement = document.getElementById("go");

// aggiungo eventi listener
/* 1mo modo: aggiungere onclick all'html (DA NON FARE!!!)
    2ndo modo: inserirlo in file js*/

goElement.addEventListener("click", function(e){
    e.target.innerHTML = "clicked";
    e.preventDefault();
    // provoca che il browser non carichi altre pagine
});
innerElement.addEventListener("click", function(e){
// e indica l'evento da gestire
    console.log("Inner " + e);
    e.stopPropagation(); // posso bloccare la propagazione
});

outerElement.addEventListener("click", function(e){
    console.log("Outer " + e);
});

function documentHandler(e){
    console.log("document " + e);
    document.removeEventListener("click", documentHandler);
    // rimuove l'evento, ma ha bisogno di conoscere la funzione
}

document.addEventListener("click", documentHandler);
```

8.3 SVG E CANVAS

HTMLELEMENT E INSERIBILI NEL DOCUMENT
VISUALIZZAZIONE GRAFICA
Scopi e modalità diversi.

SVG-> disegno vettoriale. Markup e tag HTML per elementi grafici, specifica testuale. Incorporato caratteristica HTML => tag fanno parte del DOM. API DI SCENA ovvero descrive elementi di scena quindi OGGETTI VIVI renderizzati ogni volta =>risoluzione sempre corretta.
CANVAS -> elemento DOM ma FOGLIA perchè RUSTER ovvero un array bidimensionale di pixel. Istruzioni IMPERATIVE => PRODUCONO SIDE

EFFECT-> sporco canvas. Devo ripulirlo e ridisegnare=> GESTIONE AUTONOMA CONTENUTO E NO INGRANDIBILE.

SVG elementi dinamici, CANVAS per corrispondenza di pixel o come specifiche 3D OpenJL (visualizzazione pixel).

STROKE & FILL -> dusegno poligono con oggetti parametrici.
STROKE disegna contorno figura.
FILL il riempimento.
Con SVG creo gruppi => deformazione, spostamento applicate a tutti gli oggetti del gruppo.

Codice SVG:
-OROLOGIO:

```
<html>
    <body>
        <svg width = "240" height = "240">
            <g id="clock" transform="translate(120.0,
␣␣␣␣␣␣␣␣␣␣␣␣120.0)">
                <circle id="circle" style="stroke:␣black;
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣fill:␣#f8f8f8;␣stroke-width:␣3" cx="0"
                cy ="0" r="100"></circle>
                <line x1="0" y1="0" x2="0" y2="-60"
                style="stroke:␣green;
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣stroke-width:6" id="ore"></line>
                <line x1="0" y1="0" x2="0" y2="-85"
                style="stroke:␣blue;
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣stroke-width:4" id="minuti"></line>
                <line x1="0" y1="0" x2="0" y2="-95"
                style="stroke:␣red;
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣stroke-width:2" id="secondi"></line>
            </g>
        </svg>
        <script src="clock.js"></script>
    </body>
</html>
```

```
function updateClock(){
    let now = new Date();
    document.getElementById("ore").setAttribute('transform',
    'rotate(' + now.getHours()*30.0+')');
    document.getElementById("minuti").setAttribute('transform',
    'rotate(' + now.getMinutes()*6.0+')');
    document.getElementById("secondi").setAttribute('transform',
```

```
    'rotate(' + now.getSeconds()*6.0+')');
}

function createTicks(){
    let clock = document.getElementById("clock");
    for(let minute = 0; minute < 60; minute+=5){
        let tick = document.createElementNS(
        "http://www.w3.org/2000/svg","line");
        tick.setAttribute("stroke","black");
        tick.setAttribute("stroke-width","2");
        tick.setAttribute("x1",0);
        tick.setAttribute("y1",-90);
        tick.setAttribute("x2",0);
        tick.setAttribute("y2",-100);
        tick.setAttribute("transform", 'rotate('+(minute*6.0)+')');
        clock.appendChild(tick);
    }
}

createTicks();
setInterval(updateClock,1000);
```

Mouse Follower:

```
<html>
    <head>
        <title>TODO supply a title</title>
        <meta charset="UTF-8">
        <meta name="viewport" content=
        "width=device-width, initial-scale=1.0">
    </head>
    <body>
        <div>TODO write content<p/>
        TODO write content<p/>TODO write content<p/></div>
        <script>
            function createFollower(offsetx,min,max)
            {
                 var circleDiv = document.createElement("div");
                circleDiv.innerHTML = "<svg>
                <circle cx=\"50\" cy=\"50\" r=\"20\"
                stroke=\"black\" fill=\"white\" /></svg>";
                document.getElementsByTagName("body")[0]
                .appendChild(circleDiv);

//                window.onmousemove = function(event)
                        window.addEventListener("mousemove",
                function(event)
```

```
                {
                    circleDiv.style.position = "absolute";
                    circleDiv.style.left = (event.x-50+offsetx)+'px';
                    circleDiv.style.top  = (event.y-50)+'px';
                }
                                );

                var direction = 2;
                var radius = 20;

                window.setInterval(function()
                {
                    radius = radius + direction;
                    if(radius<min) direction = 2;
                    if(radius>max) direction = -2;
                    circleDiv.firstChild.firstChild
                    .setAttribute("r",radius);
                },50);
            }

            window.onload = function(){
                                createFollower(10,20,40);
                                createFollower(100,5,30);
                        }

        </script>
    </body>
</html>
```

Pool"D:

```
function checkType(param, type){
        if(typeof type === "string"){
                if(typeof param !== type)
                        return false;
        }else{
                if(! (param instanceof type))
                        return false;
        }
        return true;
}

function assertType(param, type){
        if(! checkType(param, type))
                throw("Parameter not of type "+type.name);
        return param;
}
```

```
function getType(param){
        if(typeof param === "object")
                return param.constructor;
        else
                return typeof param;
}

function checkConfig(obj,config){
    for(let field in config){
        if(! obj.hasOwnProperty(field))
           throw("Parameter "+field+" is not ammitted for Ball");
        if(! checkType(config[field], getType(obj[field])))
           throw("Parameter "+field+" is of wrong type");
        obj[field] = config[field];
    }

    for(let checked in obj.checkers)
        if(! obj.checkers[checked](obj[checked]))
                throw ("Parameter "+checked+" not acceptable");

}


class Vector{

        x = 0;
        y = 0;

        constructor(x,y){
                if(typeof x === "number")
                        this.x = x;
                if(typeof y === "number")
                        this.y = y;
                if(x instanceof Vector){
                        this.x = x.x;
                        this.y = x.y;
                }
        }

        add(displacement){
                assertType(displacement, Vector);
                this.x += displacement.x;
                this.y += displacement.y;
                return this;
        }
```

```
        sub(displacement){
                assertType(displacement, Vector);
                this.x -= displacement.x;
                this.y -= displacement.y;
                return this;
        }

        set(point){
                assertType(point, Vector);
                this.x = point.x;
                this.y = point.y;
                return this;
        }

        mul(scale){
                assertType(scale, "number");
                if(isNaN(scale))
                        throw("Cannot␣scale␣a␣Vector␣by␣NaN");
                this.x *= scale;
                this.y *= scale;
                return this;
        }

        dot(v){
                assertType(v, Vector);
                return this.x*v.x+this.y*v.y;
        }

        len(){
                return Math.hypot(this.x,this.y);
        }

        distance(v){
                assertType(v, Vector);
                return Math.hypot(this.x-v.x,this.y-v.y);
        }

        normalize(){
                let scale = 1.0/this.len();
                this.mul(scale);
                return this;
        }

        toString(){
                return "Vector:␣[x:"+this.x+",y:"+this.y+"]";
```

```
        }

}

class Ball{

        position = new Vector();
        speed = new Vector();
        radius = 10.0;
        mass = 1.0;
        color = "#ff0000";

        checkers = {
                radius: (r)=>r>=0,
                color: (v)=>{return v.length === 7;}
        }

        constructor(config){
                checkConfig(this,config);
        }

        place(newposition){
                this.position.set(newposition);
                return this;
        }

        move(displacement){
                this.position.add(displacement);
                return this;
        }

        update(timestep){
                this.move((new Vector(this.speed)).mul(timestep));
                return this;
        }

        toString(){
                return "position: "+this.position+
            " radius: "+this.radius+" speed: "+this.speed;
        }

        touchesBall(otherBall){
                checkType(otherBall,Ball);
                return this.position.distance(otherBall.position)
            <= this.radius+otherBall.radius;
        }
```

```
        bounceWithBall(ball){
            checkType(ball,Ball);
        let delta  = (new Vector(this.position))
             .sub(ball.position).normalize();
        let impulse = 2 * (new Vector(this.speed))
         .sub(ball.speed).dot(delta) / ( (1/this.mass)
         + (1/ball.mass) );
        if(impulse < 0){
           this.speed.sub( (new Vector(delta))
             .mul( impulse / this.mass ) );
           ball.speed.add( (new Vector(delta))
             .mul( impulse / ball.mass ) );
        }
    }

        touchesSegment(segment){
            checkType(segment,Segment);
        return segment.distanceFromPoint(this.position)
         <=this.radius && segment
             .orthogonalVectorFromPoint(this.position)
             .dot(this.speed)>0.0;
    }

        bounceWithSegment(l){
            checkType(l,Segment);
        let n = l.direction();
        this.speed = new Vector(this.speed
        .dot(new Vector(n.x*n.x-n.y*n.y,2*n.x*n.y)),
         this.speed.dot(new Vector(2*n.x*n.y,n.y*n.y-n.x*n.x)));
    }

}

class Segment{

        constructor(a,b){
                this.a = assertType(a, Vector);
                this.b = assertType(b, Vector);
        }

        len(){
                return this.a.distance(this.b);
        }

        direction(){
```

```
        //let l = this.a.distance(this.b);
        //return new Vector((this.b.x-this.a.x)/l,(this.b.y-this.a.y)/l);
            return (new Vector(this.b)).sub(this.a).normalize();
    }

        distanceFromPoint(p){
            checkType(p,Vector);
        let l2 = this.a.distance(this.b);
        let t = ((p.x - this.a.x) * (this.b.x - this.a.x)
        + (p.y - this.a.y) * (this.b.y - this.a.y)) / (l2*l2);
        if (t < 0) return p.distance(this.a);
        if (t > 1) return p.distance(this.b);;
        return p.distance(new Vector( this.a.x
        + t * (this.b.x - this.a.x), this.a.y
        + t * (this.b.y - this.a.y) ));
    }

        orthogonalVectorFromPoint(p){
            checkType(p,Vector);
        let direction = this.direction();
        let projection = (new Vector(p))
        .sub(this.a).dot(direction);
        let orthogonal = (new Vector(this.a))
        .add(direction.mul(projection)).sub(p);
        return orthogonal;
    }
}

window.onload = function() {
    startGame(document.getElementById("mySvg"));
};

let currentThread = undefined;


class PoolBall extends Ball{

    static defaultRadius = 22;

    constructor(position, speed, number){
        super({
            position: position,
            radius   : PoolBall.defaultRadius,
            speed    : speed
        });
        this.number = number;
```

```
    }

    update(timeStep){
        this.move((new Vector(this.speed)).mul(timeStep));
        while(timeStep-->0)
            this.speed.mul(0.9996);
    }
}


class Pole extends Segment{

    constructor(a, b){
        super(a,b);
        this.isPole = "yes";
    }

    reset(){
        this.a = new Vector(0,0);
        this.b = new Vector(0,0);
    }
}


function startGame(svg){

    let width = svg.getBoundingClientRect().width;
    let height = svg.getBoundingClientRect().height;
    let fps = document.getElementById("fps");

    let balls = [];
    let lines = [];
    let drawables = [];

    let currentPole = new Pole(new Vector(0,0),new Vector(0,0));
    drawables.push(currentPole);

    let ballNumber = 1;
    for (let line = 0; line <= 4; line++)
    {
        let radius = PoolBall.defaultRadius * 2;
        let startY = (height - line * radius) / 2;
        let startX = (width / 2)*0.8 - line * radius;
        for (let column = 0; column <= line; column++)
        {
            let center = new Vector(startX, startY+column*radius);
```

```
            let speed = new Vector(0, 0);
            let ball = new PoolBall(center, speed, ballNumber++);
            balls.push(ball);
            drawables.push(ball);
        }
    }
    let white = new PoolBall
    (new Vector(width*0.7,height/ 2), new Vector(0,0), 0);
    balls.push(white);
    drawables.push(white);


    function addLine(line)
    {
        drawables.push(line);
        lines.push(line);
        return line;
    }
    addLine(new Segment(new Vector(67, 110),
        new Vector(67, 458)));
    addLine(new Segment(new Vector(107, 67),
        new Vector(461, 67)));
    addLine(new Segment(new Vector(535, 67),
        new Vector(890, 67)));
    addLine(new Segment(new Vector(107, 500),
        new Vector(461, 500)));
    addLine(new Segment(new Vector(535, 500),
        new Vector(890, 500)));
    addLine(new Segment(new Vector(930, 110),
        new Vector(930, 458)));

    addLine(new Segment(new Vector(30, 30),
        new Vector(30, 540))).hole = true;
    addLine(new Segment(new Vector(30, 30),
        new Vector(970, 30))).hole = true;
    addLine(new Segment(new Vector(30, 540),
        new Vector(970, 540))).hole = true;
    addLine(new Segment(new Vector(970, 30),
        new Vector(970, 540))).hole = true;


    window.addEventListener("mousedown",
        function(event)
        {
            currentPole.a =
                new Vector(event.x - svg.clientLeft,
```

```
                event.y - svg.clientTop);
        currentPole.b = new Vector(currentPole.a);
    });

window.addEventListener("mousemove",
    function(event)
    {
        if(currentPole.a.len()>0.0)
            currentPole.b =
                new Vector(event.x - svg.clientLeft,
                    event.y - svg.clientTop);
    });

window.addEventListener("mouseup",
    function(event)
    {
        if(currentPole.a.len()>0.0)
        {
            if(currentPole.len()>0)
                white.speed =
                (new Vector(currentPole.a))
                    .sub(currentPole.b).mul(0.005);
            currentPole.reset();
        }
    });


if (currentThread !== undefined)
    clearInterval(currentThread);

let currentTime = Date.now();
let lastTime = Date.now();
let count = 0;
let numSamples = 50;
let totalFps = 0;

currentThread = setInterval(
        function()
        {
            currentTime = Date.now();
            let elapsed = currentTime - lastTime;
            lastTime = currentTime;

            totalFps += 1000 / elapsed;
            if (count++ % numSamples === 0)
            {
```

```
                    fps.innerHTML = "fps: " + totalFps / numSamples;
                    totalFps = 0;
                }

                let steps = 10;
                let ministep = elapsed / steps;

                while (steps--> 0){
                    for (let b1 = 0; b1 < balls.length; b1++)
                        for (let b2 = b1 + 1; b2 < balls.length; b2++)
                            if (balls[b1].touchesBall(balls[b2]))
                                balls[b1].bounceWithBall(balls[b2]);

                    for (let b = 0; b < balls.length; b++)
                        for (let l = 0; l < lines.length; l++)
                            if (balls[b].touchesSegment(lines[l]))
                                if(lines[l].hole)
                                    balls[b]
                                    .place(new Vector(-100,-100));
                                else
                                    balls[b]
                                    .bounceWithSegment(lines[l]);

                    for (let b1 = 0; b1 < balls.length; b1++)
                        balls[b1].update(ministep);

                }

                renderAllSvg(svg, drawables);
            },
            20
            );
}


function renderAllSvg(svg, objects)
{
    objects.forEach(function (o) {
        renderDispatch(svg, o);
    });
}

var ball1 = document.getElementById("g4441");

function renderDispatch(context, object)
{
```

```
    if (object instanceof Pole)
        render_Pole(context, object);
    if (object instanceof PoolBall)
        render_Ball(context, object);
}

function render_Ball(context, object)
{
    if (!object.svgBuffer)
    {
        object.svgBuffer =
        document.getElementById("ball_" + object.number);
        let rect = object.svgBuffer.getBBox();
        object.svgBuffer.baseScale =
        object.radius * 2 / rect.width;
        object.svgBuffer.baseTranslate =
        new Vector(rect.x + rect.width / 2,
        rect.y + rect.height / 2);
        object.svgBuffer.baseTranslate
            .mul(-object.svgBuffer.baseScale);
        context.appendChild(object.svgBuffer);
    }
    let trans_x = object.svgBuffer.baseTranslate.x +
        object.position.x;
    let trans_y = object.svgBuffer.baseTranslate.y +
        object.position.y;
    let scale = object.svgBuffer.baseScale;
    object.svgBuffer.setAttribute("transform",
        "translate(" + trans_x + "," + trans_y + ")
␣␣␣␣␣␣␣␣scale(" + scale + ")");
}

function render_Pole(context, object)
{
    if (!object.svgBuffer)
    {
        object.svgBuffer = document
        .createElementNS("http://www.w3.org/2000/svg", "line");
        object.svgBuffer.setAttribute("style",
        "stroke:rgb(255,255,0);stroke-width:5");
        context.appendChild(object.svgBuffer);
    }
    object.svgBuffer.setAttribute("x1", object.a.x+0);
    object.svgBuffer.setAttribute("y1", object.a.y+0);
    object.svgBuffer.setAttribute("x2", object.b.x+0);
    object.svgBuffer.setAttribute("y2", object.b.y+0);
```

```
}
```

ESEMPIO CANVAS:

```
<html>
    <style>
        #surface{
            background-color: #ffff00;
        }
    </style>
    <body>
        <div>
            <canvas id="surface" width="240"
            height="240"></canvas>
        </div>
        <script src="canvas.js"></script>
    </body>
</html>
```

```
window.onload = function(){
    let c = document.getElementById("surface")
    .getContext("2d");
    // prende l'elemento canvas e il contesto di
    //disegno (contiene il necessario per disegnare)
    c.fillStyle = "#ff0000";
    //    impostato per tutte le azioni di riempimento successive
    c.fillRect(50,50,100,100); /
    / disegna il rettangolo passando le coordinate dei 4 punti
    c.fillStyle = "#00ff00";
    c.beginPath();
    // comincia a prendere nota di azioni di disegno da eseguire
    c.moveTo(100,100);
    // alza la penna e spostati
    c.lineTo(200,200);
    // abbassa la penna fino a queste coordinate
    c.lineTo(100,200);
    c.lineTo(100,100);
    c.lineWidth = 3; // imposta lo spessore della linea
    c.stroke(); // disegna i bordi laterali
    c.fill();
}
```